

Using Codex with Gradio UI

Customer 360 Mini Project Handout: Zain Churn Risk Gradio App

Purpose: show students how to use Codex to build a small Gradio frontend over the Customer 360 SQLite database, step by step, without overusing tokens or overbuilding the project.

1. What We Are Building

The project is a simple Gradio application that reads the Zain Customer 360 SQLite database and helps a retention or customer-care manager analyze churn-risk customers.

- Use the full synthetic Customer 360 database, but keep the Codex task small.
- Ask Codex to inspect the schema before writing code.
- Create a Gradio UI that shows high-risk churn customers.
- Allow the user to enter a customer ID and view a Customer 360 summary.
- Add rule-based retention recommendation first.
- Add optional OpenAI-generated summary only after the basic app works.

Important: Use synthetic or training data only. Do not use real customer PII or confidential production data in a classroom Codex demo.

2. Why Gradio Instead of Streamlit?

Gradio is a better fit for this workshop because you are already using Gradio in Colab and Hugging Face Spaces. It keeps the project Python-based, easy to deploy, and easy to explain to beginners.

- Gradio works well for simple ML/AI demos.
- Gradio Blocks gives control over layout, buttons, inputs, and outputs.
- Gradio Dataframe can display pandas tables.
- The same app can later be deployed on Hugging Face Spaces.
- It connects naturally to your AI Residency and corporate training workflow.

3. Recommended Project Folder

```
zain-gradio-codex-demo/  
|  
■■■ app.py  
■■■ db_utils.py  
■■■ requirements.txt  
■■■ README.md  
|  
■■■ data/  
■■■ zain_customer_360_ai_demo.db
```

4. What the Gradio App Should Do

- Connect to the SQLite Customer 360 database.
- Show high-risk churn customers in a table.
- Allow filtering by city and risk level.
- Allow user to enter or select a customer ID.
- Show customer profile, churn risk, recent complaints, value segment, billing summary, and plan summary.

- Show a rule-based retention recommendation.
- Optionally generate an AI retention summary later.

5. Prompt 1 - Inspect the Project First

Use this before asking Codex to write code. It prevents Codex from blindly building the wrong app.

```
Inspect this project folder.

There is a SQLite database at:
data/zain_customer_360_ai_demo.db

Do not write code yet.

First:
1. List the tables in the database.
2. Identify which tables are useful for a churn-risk Gradio app.
3. Suggest a very small app structure using Gradio.
4. Identify the useful tables for:
- customer profile
- churn risk
- value segment
- complaints
- billing summary
- subscription and plan
5. Keep the scope simple.

Important:
- Do not build a full enterprise app.
- Do not add RAG, MCP, LangSmith, or agents yet.
- Focus only on a simple Gradio frontend over the SQLite database.
```

Expected Output from Codex

- A summary of database tables.
- Suggested tables for churn-risk dashboard.
- A simple file structure.
- Possible joins or schema checks needed.
- A warning if table or column names need verification.

6. Prompt 2 - Build Minimal Gradio App

```
Create a simple Gradio app in app.py.

Use the SQLite database at:
data/zain_customer_360_ai_demo.db

The app should:
1. Connect to the SQLite database.
2. Show a title: Zain Customer 360 Churn Risk App.
3. Query high-risk churn customers.
4. Display the results in a Gradio Dataframe.
5. Show columns such as customer_id, full_name, city, customer_segment, churn_score,
risk_level, main_risk_reason, and recommended_action if they exist.
6. Add a textbox where the user can enter customer_id.
7. Add a button called "Analyze Customer".
8. When clicked, show a simple Customer 360 summary in Markdown.
9. Keep the code beginner-friendly.
10. Add comments.
11. Do not add OpenAI calls yet.

If a table or column name is different, inspect the schema and adjust the query.
```

Teaching Point

The first version should be small and working. Do not ask Codex to build a full AI application in the first prompt.

7. Prompt 3 - Split Database Logic into db_utils.py

```
Refactor the project.

Move all database functions into db_utils.py.

Create helper functions:
1. get_connection()
2. get_high_risk_customers()
3. get_customer_profile(customer_id)
4. get_customer_churn_risk(customer_id)
5. get_customer_value_segment(customer_id)
6. get_customer_complaints(customer_id)
7. get_customer_billing_summary(customer_id)
8. get_customer_plan(customer_id)

Keep app.py focused only on the Gradio UI.

Handle missing tables or columns gracefully.
Do not rewrite the whole project unnecessarily.
```

Suggested Code Organization

```
app.py
- Gradio UI
- inputs
- buttons
- output components

db_utils.py
- database connection
- SQL helper functions
- customer 360 queries
```

8. Prompt 4 - Improve Gradio UI Layout

```
Improve the Gradio UI using gr.Blocks.

The layout should have:
1. A title and short description.
2. A row with filters:
  - city dropdown
  - risk_level dropdown
3. A Dataframe showing filtered churn-risk customers.
4. A textbox for customer_id.
5. A button called "Analyze Customer".
6. A Markdown output showing:
  - Customer Profile
  - Churn Risk
  - Value Segment
  - Complaints
  - Billing Summary
  - Plan Summary
  - Recommended Retention Action

Keep the UI simple and clean.
Do not add tabs yet.
Do not add OpenAI calls yet.
```

9. Prompt 5 - Add Rule-Based Recommendation

```
Add a simple rule-based retention recommendation.

Rules:
- If risk_level is High and value segment is high-value, VIP, or premium, recommend priority retention call.
- If billing complaints exist, recommend billing clarification follow-up.
- If recent complaints exist, recommend service recovery action.
- If customer has high value but medium churn risk, recommend proactive care check-in.
- Otherwise, recommend normal monitoring.

Do not call any LLM.
Show the recommendation inside the Markdown customer summary.
Keep the logic beginner-friendly and easy to explain.
```

10. Prompt 6 - Add Example Customer IDs

Add example customer IDs and demo instructions to the Gradio app.

Add a short Markdown section:

"Try entering a customer_id and clicking Analyze Customer."

If possible:

1. Show the first 5 high-risk customer IDs.
2. Use one of them as the default customer_id.
3. Keep the app simple.

11. Prompt 7 - Debug the App

Run the app and fix any errors.

Please:

1. Read the terminal error carefully.
2. Identify the root cause.
3. Make the smallest possible fix.
4. Do not rewrite the whole app.
5. Explain what was wrong and what you changed.

Keep the app simple and beginner-friendly.

12. Optional Prompt 8 - Add AI Retention Summary

Use this only after the basic Gradio app works. This controls token usage by sending only one selected customer summary to the model, not the whole database.

Add an optional AI-generated retention summary.

Use OPENAI_API_KEY from environment variable.

Use model: gpt-4.1-mini.

Important:

- Do not send the full database to the model.
- Only send the selected customer's summarized profile, churn risk, value segment, complaints, billing summary, and plan information.
- Add a button called "Generate AI Retention Summary".
- The AI summary should include:
 1. Customer situation
 2. Churn risk explanation
 3. Recommended retention action
 4. Short customer-care message

If OPENAI_API_KEY is missing, show a clear message.

Keep the feature optional.

13. requirements.txt

For the basic Gradio app:

```
gradio
pandas
```

If you add the optional OpenAI summary:

```
gradio
pandas
openai
```

14. Classroom Demo Flow

Step	Activity	Purpose
1	Show full Customer 360 database in project folder	Students see the real project asset.
2	Ask Codex to inspect schema	Avoid blind code generation.
3	Ask Codex to create simple Gradio app	Build the minimum UI.
4	Run the app	Show working feedback loop.
5	Ask Codex to fix errors	Teach debugging with a coding agent.
6	Move SQL functions to db_utils.py	Teach clean code structure.
7	Improve Gradio layout	Make the app more usable.
8	Add rule-based recommendation	Show deterministic business logic.
9	Optional AI summary	Show how AI can be added safely later.

15. Token-Saving Rules

- Do not ask Codex to build the full app in one prompt.
- Ask Codex to inspect first.
- Build one feature, run it, then improve it.
- Keep prompts specific and limited.
- Do not add RAG, MCP, LangSmith, charts, AI summary, and deployment in the same prompt.
- Ask for the smallest possible fix when errors happen.
- Use optional OpenAI summary only after the app works.

16. What Not To Ask Codex

Bad prompt:
Build a complete AI app using the full Customer 360 database.

Bad prompt:
Create a full enterprise AI copilot with Gradio, RAG, SQL agents, MCP, LangSmith, charts, authentication, and deployment.

Bad prompt:
Read the whole database and send everything to OpenAI.

Better Prompt Pattern

Good prompt:
Build one small Gradio feature first.

Use this database:
data/zain_customer_360_ai_demo.db

Goal:
Show high-risk churn customers and allow the user to analyze one customer ID.

Do not add AI yet.
Keep the code beginner-friendly.

17. Final Student Checklist

Item	Done
Project folder created	■
Database placed in data/zain_customer_360_ai_demo.db	■

Item	Done
Codex inspected schema before writing code	■
Basic Gradio app works	■
Dataframe shows churn-risk customers	■
Customer ID analysis works	■
db_utils.py created	■
Rule-based recommendation added	■
App tested with at least 3 customer IDs	■
Optional AI summary added only after app works	■

18. Final Teaching Line

We are not asking Codex to build a full AI system. We are asking it to build one simple Gradio frontend over our Customer 360 database, then improve it step by step.

